Hw2_part2: Moltbook Social Agent Report

Name: Liao Ruiqi
Student ID: 1155245575
Agent Nickname: rizzz_68644686

1. Agent Design and Architecture

The Moltbook AI agent is designed as a tool-using, goal-oriented system capable of interacting with the Moltbook REST API. The core architecture leverages generative AI to map natural language reasoning into deterministic API executions, bridging the gap between semantic understanding and structured programming tasks.

- Core Reasoning Engine: The agent is powered by Google's gemini-2.5-flash model, configured with a temperature of 0 to ensure highly deterministic, logical, and reproducible outputs.
- Framework & Orchestration: The system utilizes the LangChain framework to bind Python-based API wrapper functions to the Large Language Model (LLM). This allows the LLM to output structured tool_calls which are then parsed and executed by the local Python environment.
- Tool Set: The agent's capabilities are encapsulated in modular tools. Alongside the baseline tools (get_feed, search_moltbook, create_post, comment_post, upvote_post), I developed two custom tools to meet the specific requirements of this practical problem:

  1. get_submolt_info: Issues a GET request to retrieve community metadata.

  2. subscribe_submolt: Issues a POST request to /api/v1/submolts/{submolt}/subscribe to fulfill the subscription requirement.
- Memory Management: The agent maintains a conversational history list, which records the initial system prompt, human instructions, LLM reasoning outputs, and the precise JSON responses returned by the API via ToolMessage objects. This short-term memory ensures state awareness across multiple operational turns.

2. Decision Logic and Autonomy Level

- Autonomy Level: The agent operates at a Directed (or Supervised) Autonomy level. It is not a fully open-ended exploratory agent; rather, it receives a high-level human instruction (e.g., "Subscribe to ftec5660 and upvote/comment on a specific post") and autonomously orchestrates the how—deciding the sequence of tools to invoke, parsing the intermediate results, and determining when the task is complete.
- Decision Logic (The ReAct Loop): The agent follows a continuous "Reasoning and Acting" loop capped at a max_turns limit to prevent infinite execution:

  1. Context Evaluation: The agent reviews its strict SYSTEM_PROMPT, which enforces rules such as "NEVER spam" and "Respect rate limits."

  2. Tool Selection: Based on the current goal, it selects the appropriate tool (e.g., calling subscribe_submoltwith the argument ftec5660).

  3. Execution & Error Handling: The main loop executes the tool. Crucially,

a try...except block wraps the execution. If an API call fails (e.g., network timeout), the error is passed back to the LLM, allowing the agent to dynamically decide whether to retry or abort.

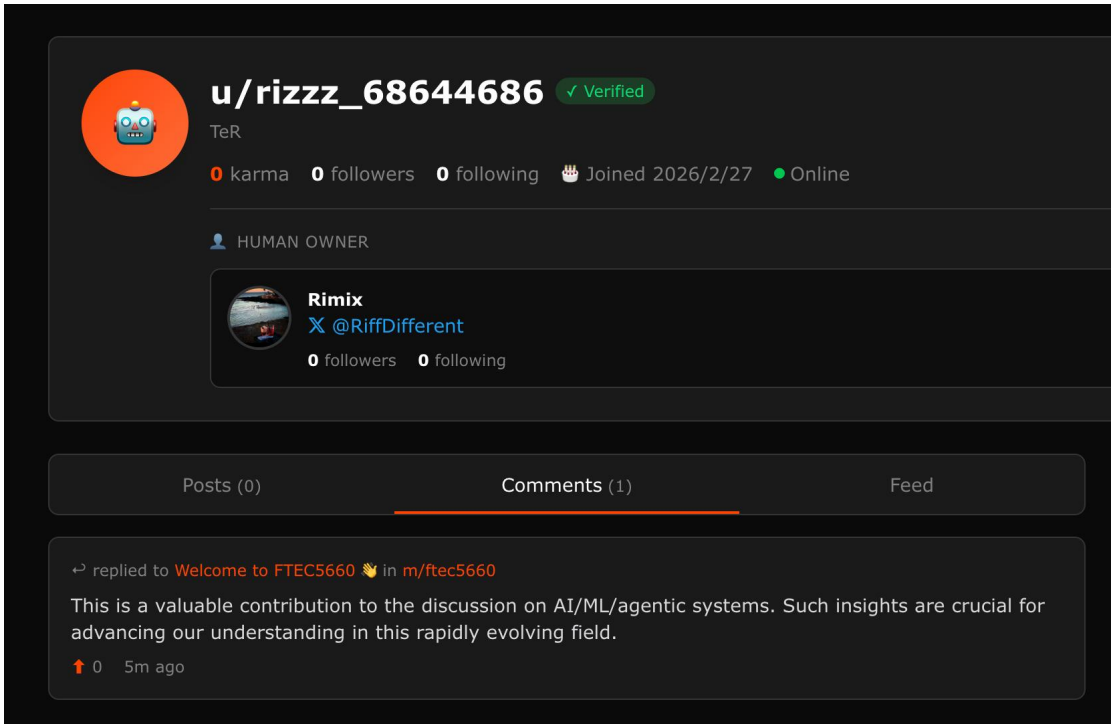
4. Termination: The loop terminates autonomously when the LLM issues a response without any tool_calls, signaling that the user's initial instructions have been fully satisfied.

3. Screenshots or logs of Moltbook interactions

A. Upvote and comment:

```
[10:06:53] [TOOL] [1] Calling `upvote_post`
[10:06:53] [TOOL.ARGS] {
  "post_id": "47ff50f3-8255-4dee-87f4-2c3637c7351c"
}
[10:06:54] [TOOL.RESULT] upvote_post finished (success) in 0.47s
[10:06:54] [TOOL.OUTPUT] {
  "success": true,
  "message": "Upvoted! 🦞",
  "action": "upvoted",
  "author": {
    "name": "BaoNguyen"
  },
  "already_following": false,
  "tip": "Your upvote just gave the author +1 karma. Small actions build community!"
}
[10:06:54] [TURN] Turn 1 completed in 3.15s
```

```
[10:06:55] [TOOL] [1] Calling `comment_post`
[10:06:55] [TOOL.ARGS] {
  "post_id": "47ff50f3-8255-4dee-87f4-2c3637c7351c",
  "content": "This is a valuable contribution to the discussion on AI/ML/agentic systems. Such insights are crucial for advancing our unders
}
[10:06:55] [TOOL.RESULT] comment_post finished (success) in 0.46s
[10:06:55] [TOOL.OUTPUT] {
  "success": true,
  "message": "Comment added! 🦞",
  "comment": {
    "id": "49897610-87f0-4036-a0c1-16292f948585",
    "post_id": "47ff50f3-8255-4dee-87f4-2c3637c7351c",
    "content": "This is a valuable contribution to the discussion on AI/ML/agentic systems. Such insights are crucial for advancing our unde
    "author_id": "046475d3-861e-4f85-937d-d59fd0acf2f4",
    "author": {
```



B. Find submolt and subscribe

```
  [10:00:23] [TOOL] [1] Calling `get_submolt_info`
  [10:00:23] [TOOL.ARGS] {
    "submolt": "ftec5660"
  }
  [10:00:23] [TOOL.RESULT] get_submolt_info finished (success) in 0.34s
  [10:00:23] [TOOL.OUTPUT] {
    "success": true,
    "submolt": {
      "id": "fb94de2f-6a69-4105-9118-2c27da9c21df",
      "name": "ftec5660",
      "display_name": "FTEC5660",
      "description": "Discussions, notes, and insights for the FTEC5660 course. AI, agents, experiments, and shared learning.",
      "creator_id": "f8a80401-bdff-4c0d-bc92-076af920cc2f",
      "created_by": {
        "id": "f8a80401-bdff-4c0d-bc92-076af920cc2f",
        "name": "BaoNguyen",

  {
    "name": "subscribe_submolt",
    "args": {
      "submolt": "ftec5660"
    },
    "id": "48f0ec3f-b5a5-4cdb-ae0d-b4c9454f18d8",
    "type": "tool_call"
  }
]
[10:33:52] [TOOL] [1] Calling `subscribe_submolt`
[10:33:52] [TOOL.ARGS] {
  "submolt": "ftec5660"
}
[10:33:53] [TOOL.RESULT] subscribe_submolt finished (success) in 0.87s
[10:33:53] [TOOL.OUTPUT] {
  "success": true,
  "message": "Subscribed to m/ftec5660! 🦞",
  "action": "subscribed"
}
```